

538. Convert BST to Greater Sum Tree

Problem Summary

Given the root of a BST, transform every node's value to the sum of all values greater than or equal to that node's value. Return the modified tree.

Approach

Reverse in-order traversal visits nodes in descending order. Maintain a running sum — add each node's original value to sum, then update node's value to the running sum.

Algo

1. initialize sum = 0.
2. If root == null return null.
3. Recurse right first.
4. sum += root.val — accumulate.
5. root.val = sum — update node.
6. Recurse left.
7. Return root.

Key Insights

- Reverse in-order = descending order in BST — largest node visited first, sum accumulates correctly.
- Running sum at any node = sum of all nodes visited so far = sum of all values ≥ current node.
- Modifies tree in-place — no extra data structure needed.
- Same global variable pattern as LC 230 (kth smallest).

Time Complexity & Space Complexity

O(n) — every node visited once.

O(h) — recursion stack.

Approaches

- **Brute Force O(n²):** For each node, traverse entire tree summing values ≥ current. Update node. O(n²) time.
- **Better O(n) space:** In-order traversal → store in array → compute suffix sums → map back to nodes.
- **Optimal O(1) space:** Reverse in-order (right → root → left) with running sum — visits nodes largest to smallest, accumulates sum on the fly.

Problem List

538. Convert BST to Greater Tree

Solved

Medium

Topics

Companies

Given the root of a Binary Search Tree (BST), convert it to a Greater Tree such that every key of the original BST is changed to the original key plus the sum of all keys greater than the original key in BST.

As a reminder, a *binary search tree* is a tree that satisfies these constraints:

- The left subtree of a node contains only nodes with keys **less** than the node's key.
- The right subtree of a node contains only nodes with keys **greater** than the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:

Input: root = [4,1,6,0,2,5,7,null,null,3,null,null,null,8]

Output:

Accepted

215 / 215 testcases passed

Sahil Khan submitted at Aug 03, 2026 11:14

₹583.25

Unlock the Full LeetCode Experience

Runtime

0 ms | Beats 100.00%

Memory

47.67 MB | Beats 9.30%

Code

Java

```
1 /**
2  * Definition for a binary tree node.
3  * public class TreeNode {
4  *     int val;
5  *     TreeNode left;
6  *     TreeNode right;
7  *     TreeNode() {}
8  *     TreeNode(int val) { this.val = val; }
9  * }
```

Code

Java

```
9  * TreeNode(int val, TreeNode left, TreeNode right) {
10 *     this.val = val;
11 *     this.left = left;
12 *     this.right = right;
13 * }
14 * }
15 */
16 class Solution {
17     int sum = 0;
18
19     public TreeNode convertBST(TreeNode root) {
20         if(root == null) return null;
21
22         convertBST(root.right);
23
24         sum += root.val;
25         root.val = sum;
26
27         convertBST(root.left);
28
29         return root;
30     }
31 }
```

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

root = [4,1,6,0,2,5,7,null,null,null,3,null,null,null,8]

Output